

WHAT'S INTERESTING?

TECHNICAL REPORT IDSIA-35-97, VERSION 1.0

Jürgen Schmidhuber
IDSIA, Corso Elvezia 36, 6900 Lugano, Switzerland
juergen@idsia.ch - <http://www.idsia.ch/~juergen>

July 14, 1997

Abstract

Interestingness depends on the observer's current knowledge and computational abilities. Things are boring if either too much or too little is known about them — if they appear either trivial or random. Interesting are unexpected regularities that seem easy to figure out. I attempt to implement these ideas in a “curious”, “creative” explorer with two co-evolving “brains”. It executes a lifelong sequence of instructions whose modifiable probabilities are conditioned on both brains — both must *agree* on each instruction. There are special instructions for comparing computational results. The brains can predict outcomes of such comparisons. If their opinions differ, then the winner will get rewarded, the loser punished. Hence each brain wants to lure the other into agreeing upon instruction subsequences involving comparisons that surprise it. The surprised brain adapts. In turn, the other loses a source of reward — an incentive to shift the focus of interest. Both brains deal with the complex credit assignment problem using the recent Incremental Self-Improvement paradigm. Extensive simulations include an example where curiosity helps to speed up external reward.

Keywords: randomness, triviality, surprise, confidence, curiosity, creativity, co-evolution, success-story algorithm, incremental self-improvement

We can learn only what we already almost know.
PATRICK WINSTON

Zwei Seelen wohnen, ach, in meiner Brust!
(*Two souls, alas! are dwelling in my breast!*)
JOHANN WOLFGANG GOETHE

1 INTUITION

Intuitively there are two types of exploration: goal-oriented and pure. The former aims at discovering solutions to externally posed tasks, the latter at increasing knowledge about the world. But how can we make the distinction precise? What seems like pure exploration at some point may help to solve problems later. For instance, old results from “pure” number theory are used in today's encryption technology.

To draw the line between goal-oriented and pure exploration I will use a reinforcement learning (RL) framework and say that exploration involves a *pure* component if there is explicit reward for knowledge growth, besides possible environmental reward.

Previous work. Existing RL approaches to pure exploration [19, 18, 30] use adaptive predictors to predict the next input given previous inputs and actions. The action-generating module gets rewarded in case of predictor failures. Thus it prefers to go to states where the predictor still can learn something. Essentially there is reward for classic information gain. For related work outside the RL context see, e.g., [6, 1, 9, 13, 15, 4]. For goal-oriented RL-based exploration see, e.g., [31, 33, 5, 25].

The “ideal ratio”. According to traditional information theory [28] noise is very informative. That is why certain previous pure explorers tend to be interested in intuitively uninteresting noise. I will try to offer a more acceptable alternative. It is inspired by Nake [14] and others who suggested that maximally interesting and aesthetically pleasing input data exhibits an ideal ratio between expected and unexpected information. Things are considered boring if they are either too random or too predictable.

What’s interesting is subjective and may change over time, because learning can make formerly unpredictable things expected, and eventually boring. I will present a simple computational framework taking this into account.

Intuitive goal. My adaptive explorer continually wants to discover new, “creative” uses of its innate sensorium and computational potential. It wants to focus on those novel things that seem easy to learn, given current knowledge. It wants to ignore (1) previously learned, predictable things, (2) inherently unpredictable ones (such as details of white noise on a screen), and (3) things that are unexpected but not expected to be easily learned (such as the contents of an advanced math textbook beyond the explorer’s current level).

What’s novel & simple? Measuring novelty requires an adaptive model of what is currently known. Estimating whether certain attempts at learning are worth the effort requires an adaptive model of what can easily be learned (of what is “simple”, given current knowledge). In this paper such models will be implicit in two partly cooperating, partly competing adaptive modules that collectively decide which computations to perform.

Basic ideas. There are two co-evolving, initially symmetric sets of modifiable parameters denoted “left brain” and “right brain”. The explorer executes a lifelong sequence of instructions selected according to probability distributions conditioned on both brains and the learner’s internal state. Instructions may change (a) environment, (b) internal state, and (c) the brains themselves (learning). I ensure that learning cannot let probabilities of instructions go to zero. Both brains are rewarded in case of external reinforcement.

There is a special instruction type “*Match?*”. Its arguments can be used to address components of the internal state and compare their current values. Once both brains have “agreed” on executing a *Match?* instruction (whose probability cannot vanish), each brain must express its opinion as to whether the addressed values will match or not. In case of different opinions one of the brains will be wrong. This one will be punished, the other rewarded. It is a zero-sum game.

To receive reward each brain will try to lure the other into agreeing on computation sequences that will surprise it. By agreeing, each brain expresses its belief that the other is in for surprises. A surprised brain will try to adapt by executing brain-modifying instructions. In turn, the other will lose a source of reward — an incentive to shift the exploration focus and try new computations. To prevent each brain from consistently outperforming the other, the currently inferior brain may also occasionally learn what the other knows by adopting the superior’s parameters.

Each brain will favor novelties that are “simple” with respect to the other’s knowledge: it will prefer quick and effortless rewards when possible. But since both brains collectively influence what they are doing none can collect quick reward by enforcing computation sequences that will make the other lose no matter what. This is a difference to previous work: a surprise corresponds to a disappointed *confident* expectation, where confidence is expressed by mutual agreement on the surprise-generating computation sequence. Details of noise, for instance, cannot be expected confidently — so they tend to get uninteresting.

Examples of learnable regularities. If the instruction set is universal and allows for arbitrary computations, then arbitrary regularities [10, 2, 29, 11] may be exploited by both brains to generate or avoid surprises. For instance, in partially predictable environments the following types of regularities may help to reliably generate matches of computational outcomes. (1) *Obser-*

vation/prediction: selected inputs computed via the environment (observations obtained through “active perception”) may match the outcomes of earlier internal computations (predictions). (2) *Observation/explanation:* memorized inputs computed via the environment may match the outcomes of later internal computations (explanations). (3) *Planning/acting:* outcomes of internal computations (planning processes) may match desirable inputs *later* computed via the environment (with the help of environment-changing instructions). (4) *“Internal” regularities:* the following computations yield matching results — subtracting 2 from 14, adding 3, 4, and 5, multiplying 2 by 6. Or: apparently, the computation of the truth value of “ n is the sum of two primes” yields the same result for each even integer $n > 2$ (Goldbach’s conjecture). (5) *Mixtures of (1-4).* For instance, raw inputs may be too noisy to be precisely predictable. Still, there may be internally computable, predictable, informative input transformations [24]. E.g., hearing the first two words of the sentence “John eats chips” does not make the word “chips” predictable, but at least it is likely that the third word will represent something edible. Examples (1-5) mainly differ in the degree to which the environment is involved in the computation processes, and in the temporal order of the computations.

Curiosity’s utility? The two-brain system is supposed to solve self-generated tasks in an unsupervised manner. But can the knowledge collected in this way help to solve externally posed tasks? Intuition suggests that the more one knows about the world the easier it will be to maximize external reward. In fact, later I will present an example where a curious system indeed outperforms a noncurious one. This does not reflect a universal law though: in general there is no guarantee that curiosity will not turn out to be harmful (e.g., by “killing the cat” [30]).

Outline. Section 2 will provide formal details of what has been said above, and explain how Incremental Self-Improvement [25, 26] can be used to deal with both brains’ complex credit assignment problems (though other machine learning paradigms may also be appropriate). Section 3 will describe details of a concrete implementation. Section 4 will describe experiments with and without external and surprise rewards. Section 5 will discuss differences to previous work and conclude.

2 MORE FORMALLY

2.1 BRAINS AND REWARDS

Two brains. The explorer’s life in environment $\mathcal{E}$ lasts from time 0 (birth) to unknown time T (death). The learner repeats the following cycle over and over ($\mathcal{A}$ denotes a set of possible instructions): select and execute $a \in \mathcal{A}$ with conditional probability

$$P(a \mid \text{LEFT}, \text{RIGHT}, \mathcal{S}, \mathcal{E}),$$

where $\mathcal{S}$ is the learner’s variable internal state with i -th real-valued component $\mathcal{S}_i$ ($i \in \{0, \dots, m-1\}$), and LEFT and RIGHT are matrices representing the learner’s brains. LEFT’s k -th variable, vector-valued column is denoted LEFT_k ; RIGHT’s is denoted RIGHT_k ($k \in \{0, \dots, n-1\}$). Instructions consume time and may change $\mathcal{E}$, $\mathcal{S}$, LEFT, RIGHT.

Inputs. Environmental inputs occasionally change special components of $\mathcal{S}$.

Primitive learning algorithms (PLAs). Instructions in $\mathcal{A}$ that can modify at least one column of LEFT or RIGHT are called *primitive learning algorithms* (PLAs). To ensure non-vanishing exploration potential PLAs may not generate brain modifications that will let an instruction probability go to zero. PLAs and other instructions can be combined to form more complex (probabilistic) learning algorithms.

Rewards. Occasionally $\mathcal{E}$ may distribute real-valued external reward equally among both brains. $R(t)$ is each brain’s cumulative external reward obtained between time 0 and time $t > 0$, where $R(0) = 0$.

Surprise rewards. $\mathcal{A}$ includes a subset of special instructions $\text{Match?}(x, y, c, d)$ with arguments $x, y \in \{0, \dots, m-1\}$ and $c, d \in \{-1, 1\}$ (each argument combination corresponds to a different *Match?* instruction in $\mathcal{A}$ — in this sense argument probabilities also depend on both brains). If

$c = d$ then no brain will be rewarded. $c = 1, d = -1$ means that LEFT bets on $\mathcal{S}_x = \mathcal{S}_y$, while RIGHT bets on $\mathcal{S}_x \neq \mathcal{S}_y$. $c = -1, d = 1$ means that LEFT bets on $\mathcal{S}_x \neq \mathcal{S}_y$, while RIGHT bets on $\mathcal{S}_x = \mathcal{S}_y$. The brain that is wrong will be punished (reward -1). The other one will be rewarded (+1).

Interpretation of surprise rewards. Instructions are likely to get executed only if both brains collectively assign them high conditional probabilities, given $\mathcal{E}, \mathcal{S}$. In this sense both must “agree” on each executed instruction of the lifelong computation process. In particular, both collectively set arguments x, y, c, d in case they decide to execute a *Match?* instruction. By setting $c \neq d$ they express that their predictions of the *Match?* outcome differ. Hence LEFT will be rewarded for luring RIGHT into agreeing upon instruction subsequences that include *Match?* instructions demonstrating that certain calculations yield results different from what RIGHT expects, and vice versa. Each brain wants to surprise the other; but each also may reduce the probability of instructions and arguments it does not like. Recall that an universal $\mathcal{A}$ allows for exploiting arbitrary regularities to trigger or avoid surprises (including the five types mentioned near the end of the introduction).

Trivial and random results. Why not provide reward if $\mathcal{S}_x = \mathcal{S}_y$ and $c = d = 1$ (meaning that both brains rightly believe in $\mathcal{S}_x = \mathcal{S}_y$)? Because then both brains would soon focus on this particular way of making a correct and rewarding prediction, and do nothing else. Why not provide punishment if $\mathcal{S}_x = \mathcal{S}_y$ and $c = d = -1$ (meaning that both brains are wrong)? Because then both brains would soon be discouraged from making any predictions at all. In case $c = d = 1$ the truth of $\mathcal{S}_x = \mathcal{S}_y$ is considered a well-known, “trivial” fact whose confirmation does not deserve reward. In case $c = d = -1$ the truth of $\mathcal{S}_x = \mathcal{S}_y$ is considered a subjectively “random”, irregular result. Nonzero surprise rewards can occur only in case both brains’ opinions differ. They reflect one brain’s disappointed *confident* expectations, and the other’s justified ones, where, by definition, “confidence” translates into “agreement on the surprising instruction sequence” — no surprise without such confidence. Note that variants of *Match?* may use predicates other than equality to generate surprises (e.g., “ $\leq$ ”, or predicates based on more complex pattern matching).

Relative reward weights? Let $RL(t)$ and $RR(t)$ denote LEFT’s and RIGHT’s respective total cumulative rewards obtained between time 0 and time $t > 0$. The sum of both brains’ surprise rewards always remains zero: we have $RL(t) + RR(t) - 2R(t) = 0$ for all t .

If we adopt the traditional hope that exploration will contribute to accelerating environmental rewards, then zero-sum surprise rewards seem to afford less need to worry about the relative weights of surprise versus other rewards than the surprise rewards of previous approaches, which did not add up to zero.

Enforcing fairness. To avoid situations where one brain consistently outperforms the other, the instruction set includes a special PLA that copies the currently superior brain onto the other. This PLA (with never-vanishing probability) will occasionally bring both brains on a par with each other.

2.2 LEARNING THROUGH INCREMENTAL SELF-IMPROVEMENT

The currently surprising brain wants to repeat similar surprises with higher probability in the future. The other wants to avoid further surprises by learning not to agree on similar computation sequences (implicitly learning what the other already knows). And it wants to be “creative” in the sense that it wants to generate new surprises for the other brain instead. In principle, both can learn by executing subsequences of instructions that include PLAs. How can we ensure that each brain indeed improves by accelerating its reward intake?

Incremental Self-Improvement (IS). In this paper I will use the recent Incremental Self-Improvement paradigm (IS) [26, 25] to deal with both brains’ complex spatio-temporal credit assignment problem. This does not necessarily mean that IS is the best way of doing so. Other RL paradigms may be appropriate, too — this paper’s basic ideas are independent of the choice of RL method. IS seems attractive, however, because (1) it does not make an explicit difference between learning algorithms and other instructions, or between learning, metalearning, metametalearning etc. (2) It properly takes into account that the success of each brain modification recursively

depends on the success of all later modifications for which it is setting the stage. (3) Its objective takes into account the entire time consumed by lifelong learning itself. (4) It is designed for quite general non-Markovian credit assignment problems in lifelong learning situations (see [26, 25] for recent IS applications). Following [25], the remainder of this section will describe LEFT’s IS-based learning process. RIGHT’s is analogous.

Checkpoints. LEFT’s entire life-time can be partitioned into time intervals separated by special times called *checkpoints*. Checkpoints are computed dynamically during the learner’s life by certain instructions in $\mathcal{A}$ executed according to the brains themselves. LEFT’s k -th checkpoint is denoted l_k . Checkpoints obey the following rules: (1) $\forall k \ 0 < l_k < T$. (2) $\forall j < k \ l_j < l_k$. (3) Except for the first, checkpoints may not occur before at least one PLA executed at least one LEFT-modification since the previous checkpoint.

Sequences of brain-modifications. SLM_k denotes the sequence of LEFT-modifications (SLM) computed by PLAs between checkpoints l_k and l_{k+1} . Since PLA execution probabilities depend on LEFT and RIGHT, the brains can in fact modify the way they modify themselves — they can devise their own probabilistic learning algorithms.

Goal. At some checkpoint t LEFT’s goal is to generate LEFT-modifications that will accelerate long-term reward intake: it wants to let $(RL(T) - RL(t))/(T - t)$ exceed the current average reward intake.

Success-story criterion (SSC). LEFT maintains a time-varying set V of past LEFT checkpoints that have led to long-term reward accelerations. Initially V is empty. Let v_k denote the k -th element of V in ascending order. SSC is satisfied at time t if either V is empty (trivial case) or if

$$\frac{RL(t) - RL(0)}{t - 0} < \frac{RL(t) - RL(v_1)}{t - v_1} < \frac{RL(t) - RL(v_2)}{t - v_2} < \dots < \frac{RL(t) - RL(v_{|V|})}{t - v_{|V|}}. \quad (1)$$

SSC demands that each checkpoint in V marks the beginning of a long-term reward acceleration measured up to the current time t .

Success-Story Algorithm (SSA). At every checkpoint of LEFT we invoke the *success-story algorithm* (SSA):

1. WHILE SSC is **not** satisfied

Undo all LEFT-modifications made since the most recent checkpoint in V .

Remove that checkpoint from V .

2. Add the current checkpoint to V .

“Undoing” a modification means restoring the preceding LEFT — this requires storing past values of LEFT-components on a stack prior to modification. (Components of LEFT and elements of V can be stored on the same stack — see section 3.) Thus each LEFT-modification that survived SSA is part of a bias shift generated after a checkpoint marking a lifelong reward speed-up: since v_j there has been more reward per time than since v_i , for $v_j > v_i$ ($v_j, v_i \in V$).

Timing SSA calls. Between two checkpoints LEFT is temporarily protected from SSA evaluations. Since the way of setting checkpoints depends on LEFT itself, LEFT can learn to influence *when* it gets evaluated. This evaluation-timing ability is important in dealing with unknown reward delays.

SSA’s generalization assumption. At the end of each SSA call, until the beginning of the next one, the only temporary generalization assumption for inductive inference is: LEFT-modifications that survived all previous SSA calls will remain useful. In absence of empirical evidence to the contrary, each surviving SLM_k is assumed to have set the stage for later successful $\text{SLM}_i, i > k$. Since life is one-way (time is never reset), during each SSA call the system has to generalize from a *single* experience concerning the utility of LEFT-modifications executed after any given previous point in time: the average reward per time since then.

Implementing SSA. Using stack-based backtracking methods such as those described in [26, 25] and section 3, one can guarantee that SSC will be satisfied right after each new SLM-start, despite interference from S , $\mathcal{E}$, and RIGHT. Although inequality (1) contains $|V|$ fractions,

SSA can be implemented efficiently: only the two SLMs on top of the stack need to be considered at any given time in an SSA call (see details in section 3). A *single* SSA call, however, may undo *many* SLMs if necessary.

What has been described for LEFT analogously holds for RIGHT. The next section will describe a particular implementation based on an assembler-like programming language similar to those used in [25, 21].

3 IMPLEMENTATION

In what follows I will describe details of the system used in the experiments.

Architecture. The internal state $\mathcal{S}$ consists of m addressable cells with addresses ranging from 0 to $m - 1$ (instead of ranging from 1 to m — this is due to a peculiarity of C, the implementation language). $\mathcal{S}_k \in \{-M, -M + 1, \dots, 0, 1, \dots, M\}$ are the current contents of the cell with address k . Instructions and arguments are encoded by a fixed set I of n integer values $\{0, \dots, n - 1\}$. For each value j in I , there is an instruction head a_j with $n_j < BS - 2$ integer-valued arguments, where BS is the instruction blocksize, and m is a multiple of BS . In the experiments I will use $BS = 9$ (there will be at most 6 arguments per instruction), $n = 24$, $m = BS(n^2 \text{ div } BS) = 576$ and $M = 100,000$. See list below for instruction syntax and semantics.

RIGHT and LEFT brains. All RIGHT_i and LEFT_i ($i \in \{0, \dots, m - 1\}$) are vectors of n positive, real values that sum up to 1.0. The k -th component of RIGHT_i (LEFT_i) is denoted $\text{RIGHT}_{i,k}$ ($\text{LEFT}_{i,k}$) for $k \in \{0, \dots, n - 1\}$. A variable *InstructionPointer* (IP) with range $\{0, \dots, m - 1\}$ always points to one of the brain pair’s columns. IP is viewed as a modifiable part of the environment.

Initialization. At system birth at time 0, all $\text{RIGHT}_{i,k}$ and $\text{LEFT}_{i,k}$ are set equal to $1/n$. All $\mathcal{S}_k$ and IP are set to zero. They will never be re-initialized again. To be able to restore modified brain columns if necessary, we introduce two initially empty stacks StackRIGHT and StackLEFT that allow for variable-sized stack entries, and the conventional *push* and *pop* operations. Instructions may change the Boolean variables BlockSSALEFT and BlockSSARIGHT (both are modifiable parts of $\mathcal{E}$ and initially FALSE at time 0).

3.1 BASIC CYCLE OF OPERATIONS

Until unknown time T (system death), the system repeats the following basic instruction cycle over and over.

1. Select instruction head $a_j \in I$ with probability $Q(IP, j)$, where

$$Q(i, j) = \frac{f(\text{RIGHT}_{i,j}, \text{LEFT}_{i,j})}{\sum_k f(\text{RIGHT}_{i,k}, \text{LEFT}_{i,k})}, \quad \text{for } i \in \{0, \dots, m - 1\}, j \in \{0, \dots, n - 1\}.$$

Here the *collective decision function* $f(x, y)$ maps real-valued x, y to real values. Given an appropriate f , each brain may “veto” instructions suggested by the other brain. Only instructions that are strongly supported by both brains are highly likely to be selected. One possibility is $f(x, y) = \min(x, y)$. In the experiments I will use $f(x, y) = xy$.

Comment: owing to peculiarities of certain instructions to be introduced below, $Q(i, j)$ will later be refined for cases where i addresses an instruction head as opposed to an argument.

2. a_j ’s $n_j \leq 6$ arguments $\in \{0, \dots, n - 1\}$ are selected according to probability distributions $Q(IP + 1, \cdot), Q(IP + 2, \cdot), \dots, Q(IP + n_j, \cdot)$ (except when $a_j = \text{Match?}$ — two of Match? ’s arguments will be treated differently — see Section 3.3.4 below).
3. Execute the selected instruction. This will consume time and may change (1) environment $\mathcal{E}$, (2) IP , (3) internal state $\mathcal{S}$; (4a) RIGHT, (4b) LEFT. If there is external reward R then set $\mathcal{S}_8 \leftarrow R$ (rewards become visible to the system in form of inputs).

4. If an input has changed one of the cell contents $\mathcal{S}_0, \mathcal{S}_1, \dots, \mathcal{S}_8$, then shift the contents of $\mathcal{S}_0, \mathcal{S}_1, \dots, \mathcal{S}_{80}$ to components $\mathcal{S}_9, \mathcal{S}_{10}, \dots, \mathcal{S}_{89}$, respectively. This results in a built-in short-term memory (long-term memory can be implemented by the system itself by executing appropriate instruction sequences).
5. If a_j did not modify IP (no conditional jump — compare instruction list below), then compute the address of the next instruction head by setting $IP \leftarrow w - w \bmod BS$. Here $w = (w_1 n + w_2) \bmod m$, where $w_1 \in \{0, \dots, n-1\}$ is selected according to probability distribution $Q(IP+7, \cdot)$, while $w_2 \in \{0, \dots, n-1\}$ is selected according to $Q(IP+8, \cdot)$.
6. Goto 1.

3.2 SSA CALLS

At a given time LEFT's current stack will either be empty or of the form $Block_1, Block_2, \dots, Block_u$, where $Block_i$ ($i \in \{1, \dots, u\}$) is of the form

$$[v_i, RL(v_i), (c_i^1, \text{LEFT}_{c_i^1}), (c_i^2, \text{LEFT}_{c_i^2}), \dots].$$

Here v_i denotes the i -th checkpoint still in LEFT's stack (compare section 2), $RL(v_i)$ denotes LEFT's cumulative reward until time v_i , c_i^k ($k \in \{1, 2, \dots\}$) is the address of the k -th LEFT-column modified in between v_i and the subsequent checkpoint, and $\text{LEFT}_{c_i^k}$ is the corresponding previous LEFT-column. The pairs $(c_i^k, \text{LEFT}_{c_i^k})$ were saved on the stack by the first PLA that changed the corresponding LEFT-columns after v_i .

The procedures $\text{SSALEFT}()$ and $\text{SSARIGHT}()$ below will be invoked by brain-modifying instructions to be discussed later.

SSALEFT():

1. If $BlockSSALEFT = \text{TRUE}$ then exit. Else:
2. Set $BlockSSALEFT = \text{TRUE}$. Set variable t equal to current time (t is a new checkpoint). Use backtracking and the information in StackLEFT to undo as many of the most recent LEFT-modifications as necessary to achieve SSC — see inequality (1) in section 2.2. Pop off the corresponding blocks in StackLEFT . This procedure guarantees that SSC will eventually be satisfied — see, e.g., [26].
3. Push t and $RL(t)$ onto StackLEFT . They are the first two elements of the next block to be pushed.

$\text{SSARIGHT}()$ is analogous to $\text{SSALEFT}()$.

3.3 SEMANTICS OF INSTRUCTION HEADS AND THEIR ARGUMENTS

In what follows, $x_1, x_2, y_1, y_2, z_1, z_2, \in \{0, \dots, n-1\}$ stand for instruction arguments selected according to probability distributions $Q(IP+1, \cdot), Q(IP+2, \cdot), \dots, Q(IP+6, \cdot)$, respectively. They are used to address state components and brain columns. For simplicity, instruction descriptions below use the following macros: $x := (x_1 n + x_2) \bmod m$; $y := (y_1 n + y_2) \bmod m$; $z := (z_1 n + z_2) \bmod m$.

3.3.1 INSTRUCTIONS OPERATING ON INTERNAL STATE ONLY

Jmpl($x_1, x_2, y_1, y_2, z_1, z_2$) : If $\mathcal{S}_x < \mathcal{S}_y$ then $IP \leftarrow z - z \bmod BS$.

Jmpeq($x_1, x_2, y_1, y_2, z_1, z_2$) : If $\mathcal{S}_x = \mathcal{S}_y$ then $IP \leftarrow z - z \bmod BS$.

Add($x_1, x_2, y_1, y_2, z_1, z_2$) : $\mathcal{S}_z \leftarrow (\mathcal{S}_x + \mathcal{S}_y) \bmod M$

Sub($x_1, x_2, y_1, y_2, z_1, z_2$) : $S_z \leftarrow (S_x - S_y) \bmod M$
Mul($x_1, x_2, y_1, y_2, z_1, z_2$) : $S_z \leftarrow (S_x S_y) \bmod M$
Div($x_1, x_2, y_1, y_2, z_1, z_2$) : if $S_y \neq 0$ then $S_z \leftarrow (S_x \text{ div } S_y) \bmod M$
Mov(x_1, x_2, y_1, y_2) : $S_y \leftarrow S_x$
Init(x_1, x_2, y_1, y_2) : $S_y \leftarrow x$

3.3.2 INSTRUCTIONS FOR INTERACTION WITH THE ENVIRONMENT

There is a point-like agent with “active perception”. It can move about in a two-dimensional environment (details of the environment will be described in Section 4). The agent’s current position is given by a pair of real values. Its current direction is given by variable *Direction* with range $[0, \dots, 2\pi]$.

MoveAgent(): Move the agent 12 unit lengths in direction *Direction*, unless an obstacle or the environment’s boundary blocks the path (the value 12 was chosen arbitrarily — neither it nor other arbitrarily chosen constants mentioned below have ever been changed during the experiments). Moving may lead to new inputs: S_0, S_1, S_2 become equal to the agent’s horizontal position, vertical position, and current direction, respectively. Here position components are rounded to integers between 1 and 1000, and directions are scaled to $[0, 100]$ and then rounded. If the point 24 unit lengths in front of the agent is within an obstacle or outside the environment, then $S_3 \leftarrow 24$ (the value 24 is chosen arbitrarily). Otherwise $S_3 \leftarrow 0$. Similarly for S_4 (input 90 degrees to the right), S_5 (behind), S_6 (to the left).

SetDirection(x_1) : $Direction = \frac{x_1}{n} 2\pi$. Compute new inputs like MoveAgent().

3.3.3 “INTROSPECTIVE” INSTRUCTIONS

GetLEFT(x_1, x_2, y_1) : $S_x \leftarrow \text{round}(M \text{ LEFT}_{x,y_1})$
GetRIGHT(x_1, x_2, y_1) : $S_x \leftarrow \text{round}(M \text{ RIGHT}_{x,y_1})$.

3.3.4 MATCH INSTRUCTIONS

Match?(x_1, x_2, y_1, y_2, c, d) : Select a value $c' \in \{0, \dots, n-1\}$ with probability $\frac{\text{LEFT}_{IP+5, c'}}{\sum_k \text{LEFT}_{IP+5, k}}$. If $c' > n/2$ then set variable $c \leftarrow 1$, else $c \leftarrow -1$. Select a value $d' \in \{0, \dots, n-1\}$ with probability $\frac{\text{RIGHT}_{IP+5, d'}}{\sum_k \text{RIGHT}_{IP+5, k}}$. If $d' > n/2$ then set variable $d \leftarrow 1$, else $d \leftarrow -1$. If $c = d$ then exit. If $S_y = S_x$ then give reward c to LEFT and reward $-c$ to RIGHT. Otherwise give reward $-c$ to LEFT and reward c to RIGHT. Set $S_7 \leftarrow c$ (surprise rewards become visible to the system in form of inputs).

Comment: arguments x, y are indexed by $IP+1, IP+2, IP+3, IP+4$, while arguments c, d are chosen according to brain columns indexed by $IP+5$. See Section 2 for Match?’s interpretation.

3.3.5 SSA-ENABLING INSTRUCTIONS

As long as *BlockSSALEFT* = TRUE, LEFT will be protected from evaluations and restorations by SSA. This is important to allow for (1) complex learning processes computing not just one but many correlated modifications of LEFT-columns, and (2) reward delays: using the *EnableSSALEFT*() instruction below, LEFT can influence when to trigger evaluations and when to wait for delayed reward.

EnableSSALEFT(x_1) : If $x_1 < 10$ (the value 10 is chosen arbitrarily) then set *BlockSSALEFT* = FALSE.

EnableSSARIGHT(x_1) : analogous to *EnableSSALEFT*.

3.3.6 PRIMITIVE LEARNING ALGORITHMS

All the brain-modifying PLAs below save the brain columns they change on a stack. The first LEFT-modifying instruction executed after some EnableSSALEFT() instruction will start a new block of LEFT-modifications, to be ended by the next EnableSSALEFT(). Analogously for RIGHT.

IncProbLEFT(x_1, x_2, y_1):

1. Call SSALEFT().
2. If there is not already some $(x, .)$ entry above the most recent (*checkpoint*, $RL(\text{checkpoint})$) pair in StackLEFT, then push the pair (x, LEFT_x) onto StackLEFT.
3. Set $\text{LEFT}_{x,k} \leftarrow \lambda \text{LEFT}_{x,k} \ \forall k \neq y_1$. Set $\text{LEFT}_{x,y_1} \leftarrow 1 - \lambda(1 - \text{LEFT}_{x,y_1})$. Here $0 < \lambda < 1$ is a real-valued constant. In the experiments I will arbitrarily use $\lambda = 0.3$. Note that step 3. includes renormalization.

IncProbLEFT has no effect though if the corresponding brain modifications would lead to at least one LEFT value below *MinProb*, a small positive real value (I use *MinProb* = 0.004).

Comment: IncProbLEFT is a PLA that permits LEFT to modify itself (provided RIGHT agrees). IncProbLEFT instructions may be used in conjunction with other instructions to form complex probabilistic learning algorithms (running between subsequent checkpoints).

DecProbLEFT(x_1, x_2, y_1): like IncProbLEFT, but step 3. is different:

3. $\text{LEFT}_{x,d} \leftarrow \lambda \text{LEFT}_{x,y_1}; \forall k \neq y_1 : \text{LEFT}_{x,k} \leftarrow \frac{1 - \lambda \text{LEFT}_{x,k}}{1 - \text{LEFT}_{x,k}} \text{LEFT}_{x,k}$.

MoveDistLEFT(x_1, x_2, y_1, y_2): like IncProbLEFT, but step 3. is different:

3. $\text{LEFT}_x \leftarrow \text{LEFT}_y$.

IncProbRIGHT(x_1, x_2, y_1): analogous to IncProbLEFT.

DecProbRIGHT(x_1, x_2, y_1): analogous to DecProbLEFT.

MoveDistRIGHT(x_1, x_2, y_1, y_2): analogous to MoveDistLEFT.

IncProbBOTH(x_1, x_2, y_1): call IncProbLEFT and IncProbRIGHT in random order.

DecProbBOTH(x_1, x_2, y_1): call DecProbLEFT and DecProbRIGHT in random order.

SSAandCopy(x_1): If $x_1 \geq 5$ then exit (the value 5 is chosen arbitrarily). Set *BlockSSALEFT* and *BlockSSARIGHT* = FALSE. Call SSALEFT() and SSARIGHT() in random order. Test if one of the brains has received more reward per time (since the most recent checkpoint still in its stack) than the other. If so:

Find those columns in the superior brain that differ from the corresponding columns in the “loser”. (In my implementation this is done efficiently by using a separate stack and a marker array tracing brain differences as they occur.) Push the loser’s different columns onto the loser’s stack (just like with IncProbLEFT and all other PLAs). Then copy the winner’s different columns onto the loser’s.

Comment: the PLA SSAandCopy() allows for ending “unfair” matches in case one brain consistently outperforms the other. SSAandCopy will make both brains identical, although their stacks will in general be quite different and reflect quite different histories of successful brain modifications.

3.3.7 BASIC CYCLE MODIFICATION

Now it is time to introduce a slight change of the basic cycle’s instruction selection procedure described in section 3.1. In case i addresses an instruction head as opposed to an argument, redefine

$$Q(i, j) = \frac{f(\text{RIGHT}_{i,j}, \text{LEFT}_{i,g(j)})}{\sum_k f(\text{RIGHT}_{i,k}, \text{LEFT}_{i,g(k)})}, \quad \text{for } i \in \{0, BS, 2BS, \dots\}, j \in \{0, \dots, n-1\}.$$

If a_i is IncProbRIGHT then $g(i)$ returns the index of a_i ’s *antagonistic* instruction head IncProbLEFT. Similarly for the other pairs of antagonistic instruction heads: (DecProbRIGHT, DecProbLEFT), (MoveDistRIGHT, MoveDistLEFT), (GetRIGHT, GetLEFT), (EnableSSARIGHT, EnableSSALEFT). This is necessary because antagonistic instructions require special treatment to achieve brain symmetry through *SSAandCopy*. For instance, suppose that LEFT’s current advantage depends on supporting some IncProbRIGHT instruction. An equal RIGHT opponent should strongly support IncProbLEFT instead.

4 EXPERIMENTS.

Previous IS experiments. It has already been shown that IS by itself can solve interesting tasks. For instance, [25] describes two agents A and B living in a partially observable 600×500 environment with obstacles. They learn to solve a complex task that could not be solved by various TD(λ) Q-learning variants [12]. The task requires (1) agent A to find and take “key A”; (2) agent A go to “door A” and open it for agent B; (3) agent B to enter through “door A”, find and take another “key B”; (4) agent B to go to another “door B” to open it (to free the way to the goal); (5) one of the agents to reach the goal. Both agents share the same design. Each is equipped with limited “active” sight: by executing certain instructions, it can sense obstacles, its own key, the corresponding door, or the goal, within up to 50 unit lengths in front of it. The agent can also move forward (up to 30 unit lengths), change its direction, turn relative to its key or its door or the goal. It can use memory (embodied by its *IP*) to disambiguate inputs. Reward is provided only if one of the agents touches the goal. This agent’s reward is 5.0; the other’s is 3.0. In the beginning, the goal is found only every 300,000 basic cycles. Through IS, however, within 130,000 trials the average trial length decreases by a factor of 60 — both agents learn to cooperate to accelerate reward intake [25].

Outline. This section’s purpose is not to elaborate on how IS can solve difficult tasks. Instead IS is used as a particular vehicle to implement the two-brain idea for preliminary attempts at studying “inquisitive” explorers. Subsection 4.1 will describe empirically observed system behavior in absence of external rewards. In Subsection 4.2 there will be additional reward for solving externally posed tasks.

Experimental details. There are $n = 24$ instructions (see section 3) and $m = BS(n^2 \text{ div } BS) = 576$ columns per brain. $M = 100,000$. Time is measured as follows: selecting an instruction head, selecting an argument, selecting one of the two values required to compute next instruction addresses, pushing or popping a brain column costs one time step. Other computations do not cost anything. This ensures that measured time is of the order of total CPU-time. For instance, selecting an instruction head plus 6 arguments plus the next *IP* address costs $1 + 6 + 2 = 9$ time steps.

Environment. Figure 1 shows a point-like agent’s 2-dimensional environment whose width is 1000 unit lengths. The large “room” in the south is a square. Its southwest corner has coordinates (0.0, 0.0), its southeast corner (1000.0, 0.0). There are infinitely many possible agent states: the agent’s current position is given by a pair of real numbers. Its initial coordinates are (50.0, 50.0), its initial direction is 0, its stepsize 12 unit lengths. Compare Subsection 3.3.2 on INSTRUCTIONS FOR INTERACTION WITH THE ENVIRONMENT.

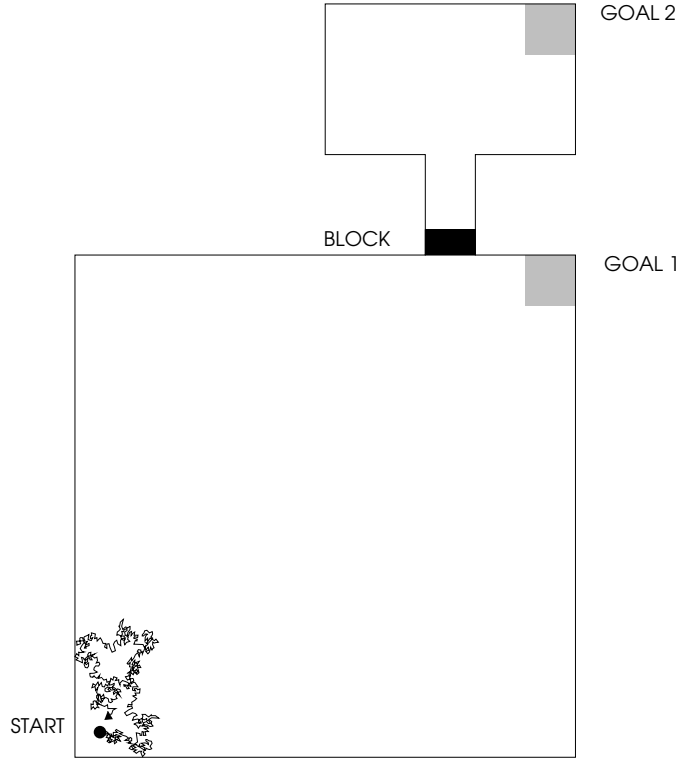


Figure 1: *Hypothetical trace of the agent near START in the southwest corner. The width of the environment is 1000 unit lengths, the agent’s stepsize 12 unit lengths. Whenever the agent hits GOAL1 or GOAL2 it gets teleported back to START (except in Experiment 1). GOAL2 provides 10 times as much external reward as GOAL1. The entrance to the small room is blocked in Experiments 1 and 2a, and open in 2b.*

4.1 EXPERIMENT 1: NO EXTERNAL REWARD.

This introductory experiment focuses on the totally unsupervised case — there is no external reinforcement for solving pre-wired tasks. Thus there also is no objective way of measuring the system’s performance. The experiment’s only purpose is to give an impression of what happens while the system is running freely. In what follows I will describe a single but rather typical run.

The entrance to the small room in Figure 1 is blocked. The system runs for 10^9 time steps corresponding to about 10^8 instructions. The system does make heavy use of its arithmetic instructions: soon the entire storage is filled with varying numbers generated as by-products of its computations.

Zero sum reward. Consider Figure 2. The derivatives of the reward plots at a given time tell which brain currently receives more reward than the other. There are long time intervals during which one of the brains dominates. The existence of zero crossings, however, shows that each brain occasionally collects sufficient negative rewards to cancel all the previously collected positive rewards, and vice versa. This means that no brain consistently outperforms the other.

Stack pointer evolution. The stack pointers reflect the number of currently valid brain column modifications. Consider Figure 3. Initially the stacks grow very quickly. Then there is a consolidation phase. Later growth resumes, but at a much more modest level — it becomes harder and harder for each brain to acquire additional “probabilistic knowledge” to outwit the other. Sometimes SSA pops off a brain’s entire stack (Figure 3’s temporal resolution is too low to show this), but in such cases the stack pointer soon reaches its old value again. This is partly due to *SSAandCopy* instructions copying the currently superior brain onto the other to enforce fair matches.

Reward acceleration. Even in later stages both brains are able to accelerate their long-term average reward intake again and again, despite the fact that the sum of all rewards remains zero. As each brain collects a lot of negative reward during its life, it continually pops appropriate checkpoints/modifications off its stack such that the resulting histories of surviving brain modifications correspond to histories of less and less negative reward/time ratios. Recall the occasional popping off of the entire stack.

Evolution of instruction frequencies. Consider Figure 4. Although the sum of all surprise rewards remains zero, *Match?* instructions soon are among the most frequent types. Other instructions also experience temporary popularity, often in form of sharp peaks beyond the plot’s resolution. Usually, however, the brain that does not profit from them learns to put its veto in.

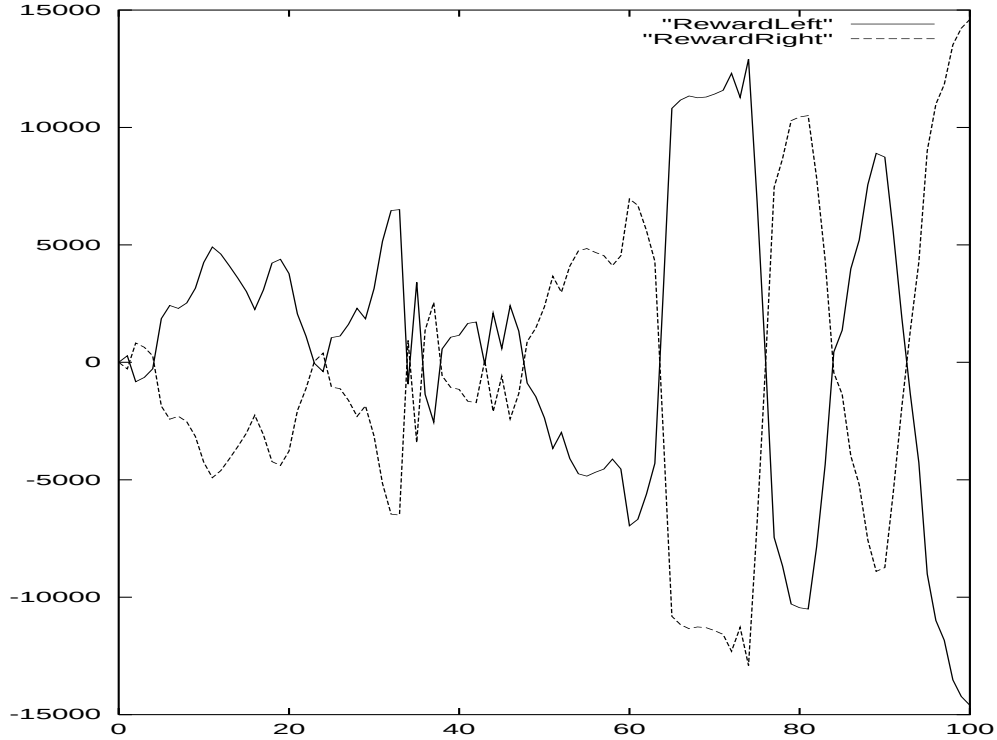


Figure 2: *Experiment 1 (no external rewards): symmetric evolution of both brains’ cumulative surprise rewards during the first billion time steps, sampled at intervals of 10 million steps.*

Interpretation difficulties. Interestingness depends on current knowledge and computational abilities. These are very different for human observers and my particular implementation. It seems hard to trace and understand the system’s millions of self-surprises and self-modifications. Figure 4, for instance, provides only very limited insight into the nature of the complex computations carried out. It plots frequencies of selected instruction types but ignores the corresponding arguments and *InstructionPointer* positions. It does not reflect that for a while computations may focus on just a few brain columns and storage cells, until this gets “boring” from the system’s point of view. Much remains to be done to analyze details of the system’s complex dynamics.

Creativity’s utility? Can the probabilistic knowledge collected by a “creative” two-brain system (solving self-generated tasks in an unsupervised manner) help to solve externally posed tasks? The following experiment provides a simple example where this type of curiosity indeed helps to achieve faster goal-directed learning in case of occasional non-zero environmental reward.

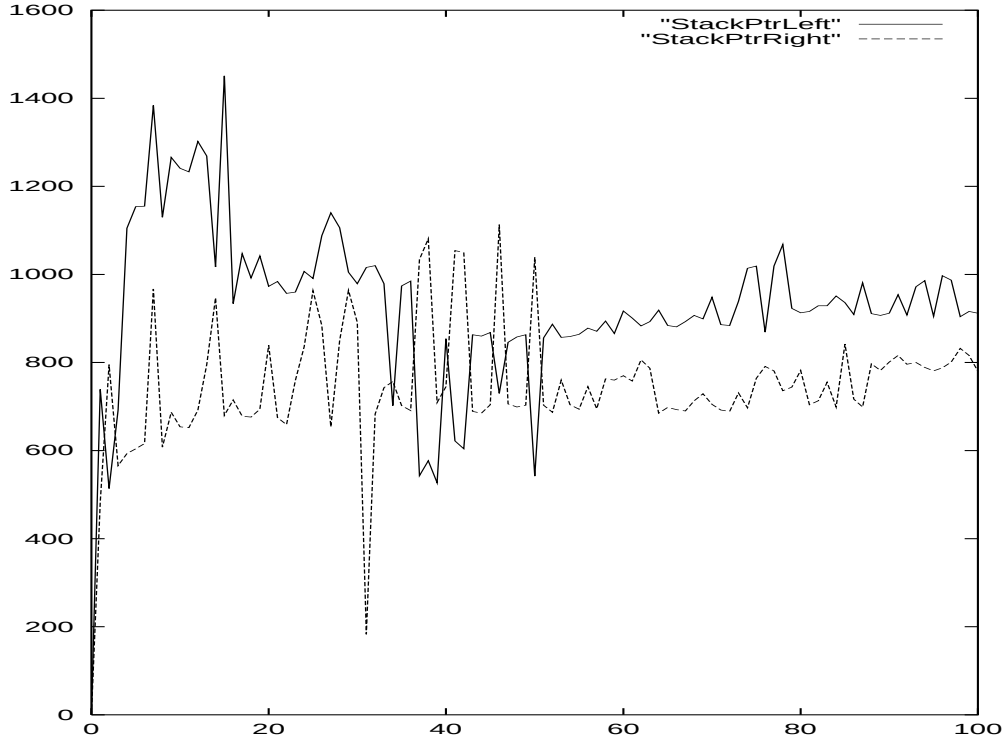


Figure 3: *Experiment 1: evolution of both brains' stack pointers during the first billion time steps, measured at intervals of 10 million steps.*

4.2 EXPERIMENT 2: ADDITIONAL EXTERNAL REWARD.

4.2.1 EXPERIMENT 2A

Consider Figure 1. The entrance to the small room is blocked. Whenever the point-like agent moves into the 100×100 northeast corner (GOAL1) of the 1000×1000 field it receives external reward 100.0 and is immediately teleported back to start position (50.0, 50.0); its direction is reset to 0.0.

Solution. GOAL1 can be reached by a simple stochastic policy making the agent head in the northeast direction. Once the system somehow has fixed the agent's direction in an optimal way, the shortest path requires about 100 calls of *MoveAgent* (due to the agent's limited stepsize). We will see that the major difficulties in *learning* such a policy are due to the system's extremely low initial bias towards solving this task, and the extremely rare occurrences of rewarding training instances.

Random behavior results. With its brain-modifying capabilities being switched off (PLAs such as $\text{IncProbLEFT}(x_1, x_2, y_1)$ have no effect), the system exhibits random behavior according to its maximum entropy initialization. This behavior leads to about 5 visits of GOAL1 per 10 million time steps.

Initial bias. Following Geman and Bienenstock [7], the learning system's bias towards solving the task is low, while variance is high. There is a confusing choice of $2 \times 576 = 1152$ brain columns (many more than needed for solving the task), all being potential candidates for modifications. The two brains initially have no idea that two of the 23 instruction types (namely *MoveAgent* and *SetDirection*) are particularly important. Furthermore, brain modifications can be computed only by appropriate instruction subsequences (including PLAs) generated according to the brain columns themselves. Initially all instructions are extremely stochastic, however, partly due to the many arguments and addresses to jump to. Reducing this "free parameter overkill" by using

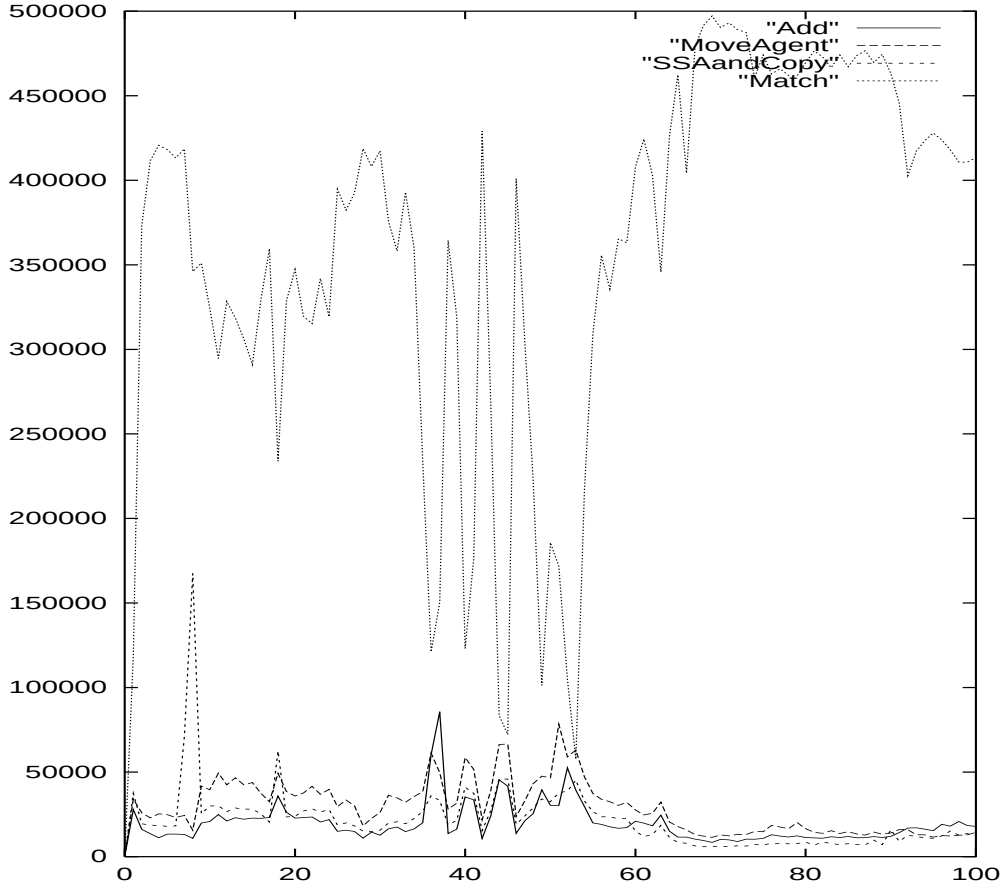


Figure 4: *Experiment 1: execution frequencies of selected instruction types during the first billion time steps, sampled at intervals of 10 million steps.*

smaller brains would be equivalent to inserting more *a priori* knowledge about the task.

Comparison. I compare the performance of two learning systems. One is curious/creative in the sense described in the previous sections, the other is not. The latter is like the former except that its *Match?* action has no effect — external reward is the only type of reward. Ten simulations are conducted for each system. Each simulation takes 200 million time steps. During the tenth and the twentieth 10^7 time step interval I count how often the agent visits GOAL1.

Rare training examples. Since external reward initially occurs only every 2 million time steps on average, goal-relevant training examples are rare.

Results. Table 1 lists the plain and the curious systems’ goal visits per 10^7 time steps after half their life-times, and near death. All 10 results are given due to the high variance. The curious system’s results tend to be more than an order of a magnitude better than the plain’s, which tend to be more than an order of a magnitude better than the random’s.

Figures 5, 6, 7 show details of the particularly successful simulation 1. We observe that total reward fluctuations due to surprise rewards appear negligible, although they seem important for overall performance improvement (compare Table 1). Between 20 and 30 million steps there are lots of *Match* and *SSAandCopy* actions. Soon afterwards the instruction types *Move* and *SetDirection* start to dominate. Around 130 million steps their frequencies suddenly rise dramatically. The final breakthrough is achieved shortly thereafter. The brains’ first 100 columns after simulation 1 are shown in Figure 8. Both brains are almost identical. The probability mass of many columns is concentrated in a single value. Additional inspection revealed, however, that some of the most

Simulation	plain: half	curious: half	plain: final	curious: final
1	4	899	9	15774
2	4	0	4	0
3	3	204	15	381
4	124	558	1036	3150
5	399	539	1000	2073
6	9	1024	82	2937
7	9	1024	123	1181
8	6	891	20	1561
9	58	89	324	446
10	62	5603	476	9963
Average	68	1083	309	3746

Table 1: *Experiment 2a: 10 simulations, each taking $2 * 10^8$ time steps. The table lists goal visits achieved by plain and curious systems during the tenth (half life-time) and the final 10^7 time step interval. Random behavior leads to about 5 visits per 10^7 time steps. Curiosity often seems to help a lot. But note the second simulation’s “outlier”.*

frequently used instruction addresses point to maximum entropy distributions.

Possible explanation. How can curiosity/creativity help? In early system life external rewards occur very rarely. During this time, however, there are many surprise reward-oriented instruction subsequences. Possibly they provide the system with useful fragments of “probabilistic knowledge” about consequences of its innate instructions. For instance, to repeatedly surprise itself it may construct little “probabilistic subprograms” involving highly probable jumps to appropriate brain column addresses. Once it has figured out how to increase the likelihood of certain jumps it may find it easier to solve external tasks that also require jumps.

Details of how the system actually came up with the final matrices remain quite unclear, however. More experimental analysis is necessary to better understand how solving external tasks can benefit from pure curiosity.

The outlier. Note the curious system’s outlier in the second simulation. Inspection revealed that during the entire simulation the system never reached the goal at all — there wasn’t a single training example concerning external reward. This outlier reflects curiosity’s potential drawback: it may focus attention on computations that do not have anything to do with external reinforcement.

Runtimes. To some readers the runtimes of several 100 million time steps may seem large. Note, however, that the number of training examples (trials) is much smaller because the goal is hit very rarely, especially in the beginning of the learning phase. Some trials take millions of time steps. The following simulations will require even longer runtimes.

4.2.2 EXPERIMENT 2B

I keep the basic set-up from Experiment 2a, but open the entrance to the small room in Figure 1 by removing the block. Whenever the point-like agent moves into the small room’s 100×100 northeast corner (GOAL2) it receives external reward 1000 (ten times as much as for reaching GOAL1) and is teleported back to start position (50.0, 50.0); its direction is reset to 0.0.

The idea is: knowledge collected by solving the simpler task (reaching GOAL1) may help to solve the more difficult but also more rewarding task (reaching GOAL2). Note that *both* tasks are solvable by similar stochastic policies making the agent head northeast. To receive a lot of reward, however, the agent must avoid GOAL1 on its way to GOAL2 — otherwise it will be teleported back home.

This set-up involves an obvious goal-directed exploration component: it is likely that the system will keep an exploration strategy that has helped to improve a policy for reaching GOAL1

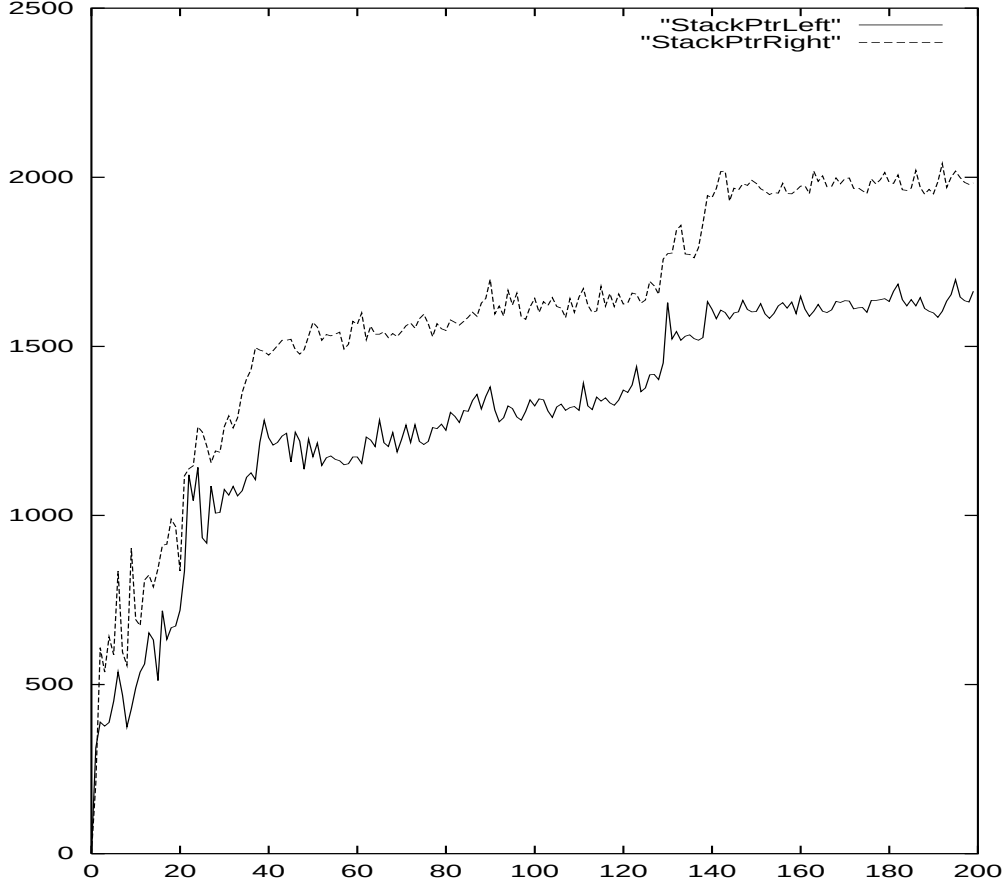


Figure 5: *Experiment 2a (additional external reward): stack pointer evolutions during the first 200 million time steps of the particularly successful simulation 1, sampled at million step intervals. Compare Figures 6 and 7.*

(successful exploration strategies have an “evolutionary advantage”). This strategy may later also help to improve a policy for reaching GOAL2 — in principle, IS can use experience with exploration strategies to evaluate and refine them.

Random behavior results. With its brain-modifying capabilities being switched off (PLAs such as $\text{IncProbLEFT}(x_1, x_2, y_1)$ have no effect), the system exhibits random behavior according to its maximum entropy initialization. On average this behavior leads to less than 1 visit of GOAL2 per 10 million time steps.

Comparison. Again I compare the performance of a “plain” and a “curious” system. Again the former is like the latter except that its *Match?* instructions have no effect. Ten simulations are conducted for each system. Each simulation takes 4 billion time steps. I keep track of how often the agent reaches GOAL2 per 10^7 time steps.

Results. Figure 9 plots average goal visit frequencies during the first 500 million time steps. During this period, the curious system’s results are clearly better than the plain’s, and its performance improves much faster.

As more simulation time is spent, however, initial differences tend to level out. Figure 10 plots average goal visit frequencies during the entire 4 billion steps. As goal-oriented training examples become more and more frequent, we see that performances of plain and curious systems eventually reach comparable levels — the former even becomes a bit better than the latter.

Advantage in case of rare rewards? For this particular experiment, self-generated surprise

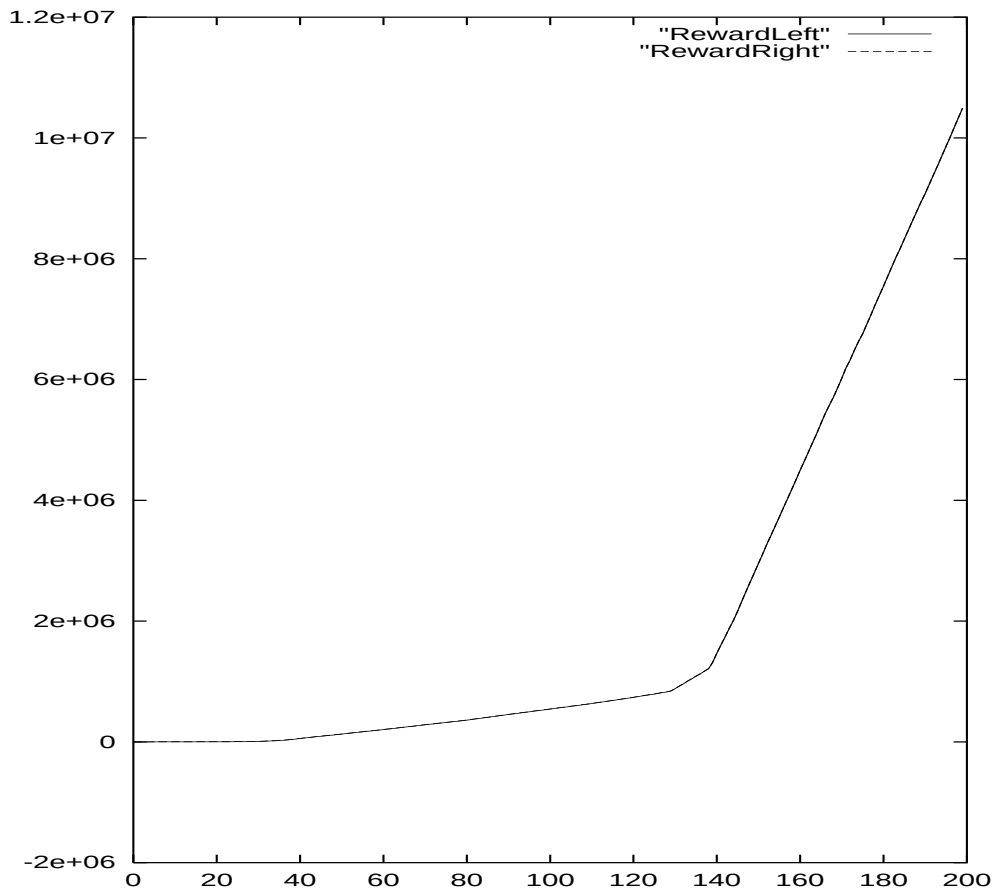


Figure 6: *Experiment 2a: evolution of both brains’ cumulative rewards during the first 200 million time steps of simulation 1, sampled at million step intervals. Plot resolution is too low to show that both curves are not identical — fluctuations due to surprise rewards seem negligible. Still, surprise rewards seem essential for significant performance improvement — compare Table 1. The final breakthrough occurs around 140 million steps. Compare Figures 5 and 7.*

rewards appear to boost initial external reward. Sufficient training time, however, cancels out the initial advantage. Could it be that curiosity is particularly useful as long as external reward is extremely rare and little has already been learned? This may seem intuitively plausible: in the beginning performance tends to be so bad that time spent on extensive exploration seems to be a good investment with little downside potential — things cannot get much worse. Once the system’s strategy is rather efficient and yields frequent external rewards, however, additional progress depends on fine-tuning it. In this stage surprise reward-oriented curiosity may distract more than it helps — it tends to consume time without necessarily contributing much to optimizing goal-directed trajectories. Many additional experiments are necessary, however, to understand whether the above is a typical result or not.

5 FINAL REMARKS

Previous work on IS (e.g., [25, 26]) implicitly focused on goal-oriented exploration — in principle IS can learn to change its exploration strategy if this turns out to accelerate external reward in the long run. This paper’s IS implementation, however, involves an additional *pure* exploration component besides the goal-oriented one. Part of the learner receives internal reward for pointing

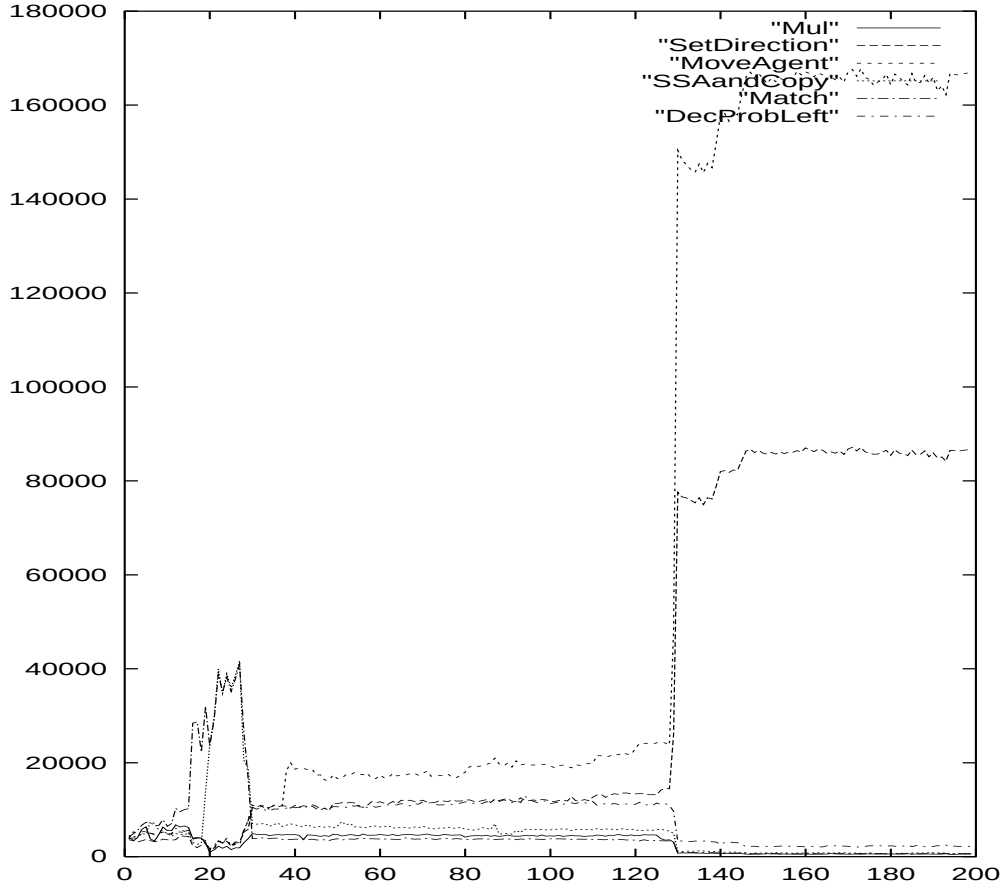


Figure 7: *Experiment 2a: execution frequencies of selected instruction types during the first 200 million time steps of simulation 1, sampled at million time step intervals. Between 20 and 30 million steps there are many Match and SSAandCopy instructions. Soon afterwards Move and SetDirection start to dominate. The final breakthrough is achieved 10 million steps after their frequencies increase dramatically around 130 million steps. Compare Figures 5 and 6.*

out something another part did not know but thought it knew. The surprised part suffers to the extent the surprising part benefits — the sum of all internal rewards remains zero. The learner is “interested” in “creative” computations leading to unexpected results, while simultaneously trying to make formerly surprising things predictable and boring. It does not care much for irregular noise rich with Shannon-information [28]. Instead it prefers easily learnable regularities, taking into account the costs of gaining information in an RL framework.

From each brain’s perspective an instruction subsequence is “novel” as long as its outcome surprises the other. Since the surprised brain will eventually figure out what is going on, there will be incessant pressure to create new novelties. What is the use of such an inquisitive system’s computations? Curiosity’s long-term justifiability depends on whether knowledge growth will eventually support goal-oriented behavior. When will this be the case? The question is reminiscent of G. H. Hardy’s toast on “pure” mathematics — the kind that “would never be of any use to anyone” ([3], p. 185). History teaches us, however, that it is hard to decide which math will be useless forever.

What’s old. This paper’s approach draws inspiration from several sources. For instance, the two-brain system is based on two co-evolving modules. Co-evolution of competing strategies, however, is nothing new. See, e.g., [8, 16] for interesting examples. Also, the idea of improving a

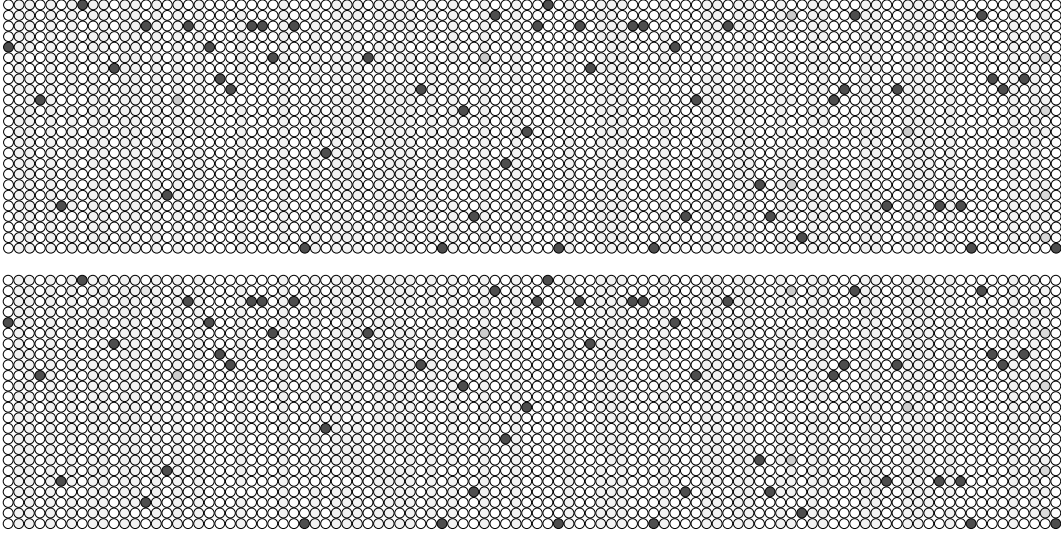


Figure 8: *Experiment 2a: LEFT’s (top) and RIGHT’s first 100 (of 576) probability distributions after simulation 1. Grey scales indicate probability magnitudes (white = close to 0, black = close to 1). The probability mass of many (but not all) columns is concentrated in a single value. Both brains are almost identical due to SSAandCopy PLAs. Their stacks are quite different though.*

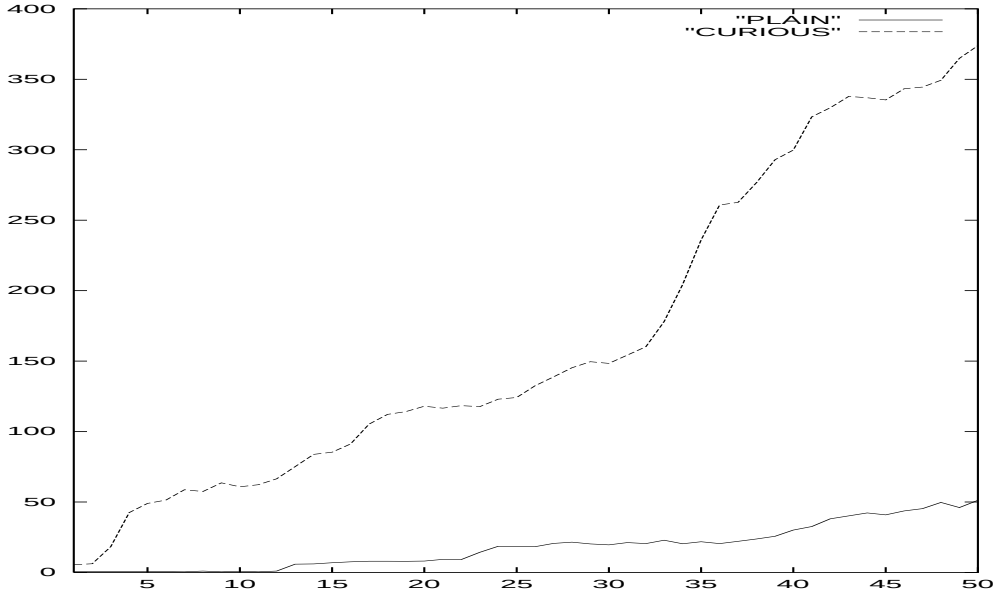


Figure 9: *Experiment 2b: comparison of plain and curious systems. The plot shows visits of GOAL2 during the first 500 million time steps, sampled at intervals of 10^7 steps (averaged over 10 simulations; random behavior yields less than 1 visit per 10^7 steps). Compare Figure 10.*

learner by letting it play against itself is ancient. See, e.g., [17, 32].

Even the idea of unsupervised learning through co-evolution of predictors and modules trying to escape the predictions is nothing new — it has been used extensively in our previous work on unsupervised sensory coding [20, 27, 24, 23]. Finally, co-evolutionary methods translating mismatches between reality and expectations into reward for “curious”, exploring agents are not new either — see our previous work on “pure” RL-based exploration [19, 18, 30]. So, what’s new?

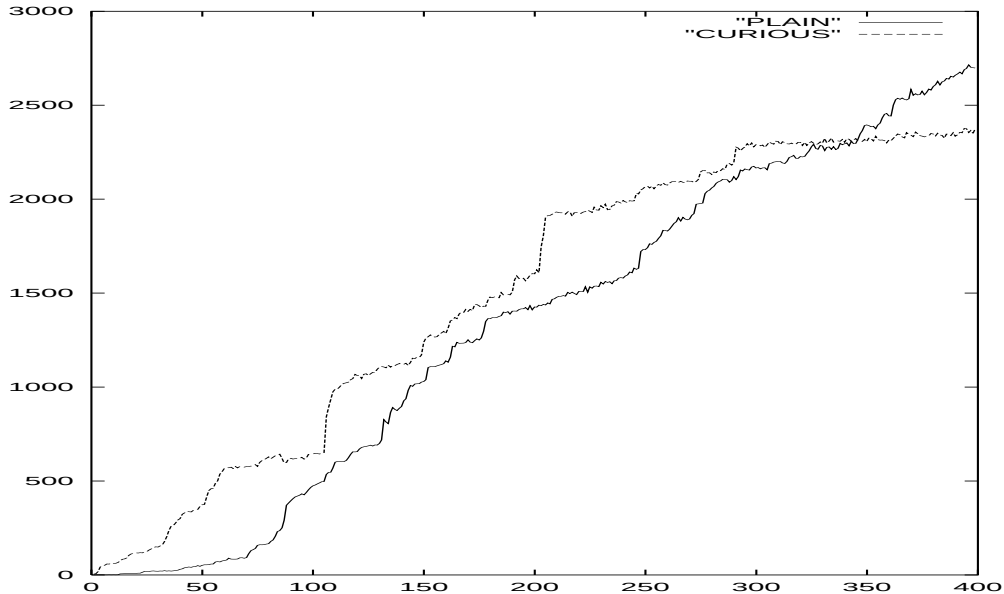


Figure 10: *Experiment 2b: visits of GOAL2 during the entire 4 billion time steps, sampled at intervals of 10^7 steps (averaged over 10 simulations). Initial advantages of curiosity are lost as more and more goal-oriented training examples become available. Compare Figure 9.*

Competitors collectively decide. Novel is the idea that both adaptive modules equally influence the probability of each executed instruction/computation. This (1) allows for a straightforward way of making both modules equally powerful (by copying the currently superior one onto the other), and (2) prevents each module from being able to enforce computations that will make the other lose no matter what it tries. For instance, details of white noise on a screen are inherently unpredictable, but none of the two opponents may exploit this to generate surprises if the other does not “agree”. And it will agree only as long as it suspects that there is a regularity in the white noise that the other does not know yet. The precondition of a surprise is that the surprised module has expressed its confidence in a different outcome of the surprising computation sequence by participating in the collective decision process.

Generality. Another novel aspect is the general setting. Instead of being limited to Markovian domains and simple reactive strategies such as approaches in [18, 30], this paper’s set-up allows for quite arbitrary domains and computations. This is made possible by the recent IS paradigm [25, 26]. There is no essential limit (besides computability) to the nature of the regularities that may be exploited to generate surprises. Neither is there an essential limit to the nature of the learning processes that can make formerly surprising regularities predictable and boring. There may be RL schemes even more general than IS, but this is beyond the scope of this paper.

Other notions of regularity. Note that my notion of “simple regularities” differs from, e.g., Kolmogorov complexity theory’s [10, 2, 29, 11, 21]. There an object is called simple relative to current knowledge x if the size of the shortest algorithm computing it from x is small. The algorithm’s computation time is ignored, as are constant factors reflecting Kolmogorov complexity’s machine independence. The current paper, however, takes both into account.

No prewired ratio between expected and unexpected info. As the explorer’s knowledge about its environment and computational abilities expands, it keeps balancing on the thin, dynamically changing line between the subjectively random and the subjectively trivial. Unlike Nake and other authors he cites [14], I do not suggest a predefined optimal ratio between known and unknown information. Instead, the two cooperating/competing brains dynamically, implicitly determine this ratio as they keep trying to surprise each other.

Interestingness and “beauty”. A recent paper attempts at explaining “beauty” with the

help of complexity theory concepts [22]. It argues that something “beautiful” needs not be “interesting”. Its formalism predicts that the “most beautiful” object from a set of objects satisfying certain specifications is the one that can be most easily computed from the subjective observer’s input coding scheme. Interestingness in the current paper’s sense, however, also takes into account whether the computational result is expected or not. Something that is both “beautiful” and already known may be quite boring — “beauty” needs to be unexpected to awake interest.

Future work. The principles put forward in this paper are rather general. They can be used in implementations different from Section 3’s — there are many possible schemes for letting two adaptive modules collectively compute probabilities of action sequences. For instance, probabilistic recurrent neural nets instead of assembler-like languages may be used to implement the two-brain principle. Furthermore, although IS is a rather general RL paradigm, it may be possible to develop more general ones. In that case I would like to combine them with the two-brain idea.

The programming language used in the experiments is designed to allow for fairly arbitrary computations/explorations and learning processes. To make progress towards analyzing “inquisitive” explorers, however, one will probably have to study alternative systems with less computational power and less general RL paradigms but more accessible dynamics. On the other hand, it will also be interesting to study a curious learner’s performance in case of more difficult tasks and more powerful primitive instructions with more bias towards solving the task. Note that PLAs can be almost anything: neural net algorithms, Bayesian analysis algorithms, etc.

All of this may help to understand under which conditions curiosity can actually improve goal-directed learning. It will always be possible, however, to design environments where “curiosity kills the cat” [30], or at least has negative influence on external performance. For instance, as exemplified by simulation 2 of Experiment 2a, curiosity may occasionally prevent discovery of external reward sources.

6 ACKNOWLEDGMENTS

I would like to thank Jieyu Zhao (supported by SNF grant 21-43’417.95 “Incremental Self-Improvement”), Nic Schraudolph, Rafał Śalustowicz, Sepp Hochreiter, Marco Wiering, and Luca Gambardella for valuable discussions and comments on a draft of this paper.

References

- [1] E. B. Baum. Neural nets that learn in polynomial time from examples and queries. *IEEE Transactions on Neural Networks*, 2(1):5–19, 1991.
- [2] G.J. Chaitin. On the length of programs for computing finite binary sequences: statistical considerations. *Journal of the ACM*, 16:145–159, 1969.
- [3] A. C. Clarke. *The ghost from the grand banks*. Orbit books, 1991.
- [4] D. A. Cohn. Neural network exploration using optimal experiment design. In J. Cowan, G. Tesauero, and J. Alspector, editors, *Advances in Neural Information Processing Systems 6*, pages 679–686. San Mateo, CA: Morgan Kaufmann, 1994.
- [5] P. Dayan and T. J. Sejnowski. Exploration bonuses and dual control. *Machine Learning*, 25:5–22, 1996.
- [6] V. V. Fedorov. *Theory of optimal experiments*. Academic Press, 1972.
- [7] S. Geman, E. Bienenstock, and R. Doursat. Neural networks and the bias/variance dilemma. *Neural Computation*, 4:1–58, 1992.
- [8] D. Hillis. Co-evolving parasites improve simulated evolution as an optimization procedure. In C.G. Langton, C. Taylor, J. D. Farmer, and S. Rasmussen, editors, *Artificial Life II*, pages 313–324. Addison Wesley Publishing Company, 1992.
- [9] J. Hwang, J. Choi, S. Oh, and R. J. Marks II. Query-based learning applied to partially trained multilayer perceptrons. *IEEE Transactions on Neural Networks*, 2(1):131–136, 1991.

- [10] A.N. Kolmogorov. Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1:1–11, 1965.
- [11] M. Li and P. M. B. Vitányi. *An Introduction to Kolmogorov Complexity and its Applications*. Springer, 1993.
- [12] L.J. Lin. *Reinforcement Learning for Robots Using Neural Networks*. PhD thesis, Carnegie Mellon University, Pittsburgh, January 1993.
- [13] D. J. C. MacKay. Information-based objective functions for active data selection. *Neural Computation*, 4(2):550–604, 1992.
- [14] F. Nake. *Ästhetik als Informationsverarbeitung*. Springer, 1974.
- [15] M. Plutowski, G. Cottrell, and H. White. Learning Mackey-Glass from 25 examples, plus or minus 2. In J. Cowan, G. Tesauro, and J. Alspecter, editors, *Advances in Neural Information Processing Systems 6*, pages 1135–1142. San Mateo, CA: Morgan Kaufmann, 1994.
- [16] J. B. Pollack and A. D. Blair. Why did TD-Gammon work? In M. C. Mozer, M. I. Jordan, and S. Petsche, editors, *Advances in Neural Information Processing Systems 9*, pages 10–16. MIT Press, Cambridge, 1997.
- [17] A. L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal on Research and Development*, 3:210–229, 1959.
- [18] J. Schmidhuber. Curious model-building control systems. In *Proc. International Joint Conference on Neural Networks, Singapore*, volume 2, pages 1458–1463. IEEE, 1991.
- [19] J. Schmidhuber. A possibility for implementing curiosity and boredom in model-building neural controllers. In J. A. Meyer and S. W. Wilson, editors, *Proc. of the International Conference on Simulation of Adaptive Behavior: From Animals to Animats*, pages 222–227. MIT Press/Bradford Books, 1991.
- [20] J. Schmidhuber. Learning factorial codes by predictability minimization. *Neural Computation*, 4(6):863–879, 1992.
- [21] J. Schmidhuber. Discovering neural nets with low Kolmogorov complexity and high generalization capability. *Neural Networks*, 1997. In press.
- [22] J. Schmidhuber. Low-complexity art. *Leonardo, Journal of the International Society for the Arts, Sciences, and Technology*, 30(2):97–103, 1997.
- [23] J. Schmidhuber, M. Eldracher, and B. Foltin. Semilinear predictability minimization produces well-known feature detectors. *Neural Computation*, 8(4):773–786, 1996.
- [24] J. Schmidhuber and D. Prelinger. Discovering predictable classifications. *Neural Computation*, 5(4):625–635, 1993.
- [25] J. Schmidhuber, J. Zhao, and N. Schraudolph. Reinforcement learning with self-modifying policies. In S. Thrun and L. Pratt, editors, *Learning to learn*. Kluwer, 1997. To appear.
- [26] J. Schmidhuber, J. Zhao, and M. Wiering. Shifting inductive bias with success-story algorithm, adaptive Levin search, and incremental self-improvement. *Machine Learning*, 28:105–130, 1997. In press.
- [27] N. Schraudolph and T. J. Sejnowski. Unsupervised discrimination of clustered data via optimization of binary information gain. In Stephen José Hanson, Jack D. Cowan, and C. Lee Giles, editors, *Advances in Neural Information Processing Systems*, volume 5, pages 499–506. Morgan Kaufmann, San Mateo, 1993.
- [28] C. E. Shannon. A mathematical theory of communication (parts I and II). *Bell System Technical Journal*, XXVII:379–423, 1948.
- [29] R.J. Solomonoff. A formal theory of inductive inference. Part I. *Information and Control*, 7:1–22, 1964.
- [30] J. Storck, S. Hochreiter, and J. Schmidhuber. Reinforcement driven information acquisition in non-deterministic environments. In *Proceedings of the International Conference on Artificial Neural Networks, Paris*, volume 2, pages 159–164. EC2 & Cie, Paris, 1995.
- [31] R. S. Sutton. Integrated architectures for learning, planning and reacting based on dynamic programming. In *Machine Learning: Proceedings of the Seventh International Workshop*, 1990.

- [32] G. Tesauro. TD-gammon, a self-teaching backgammon program, achieves master-level play. *Neural Computation*, 6(2):215–219, 1994.
- [33] S. Thrun and K. Möller. Active exploration in dynamic environments. In D. S. Lippman, J. E. Moody, and D. S. Touretzky, editors, *Advances in Neural Information Processing Systems 4*, pages 531–538. San Mateo, CA: Morgan Kaufmann, 1992.